

High-Performance n-Dimensional Gaussian Splatting in Pure PyTorch: A CUDA-Free Implementation Guide for Scientific Imaging

Introduction

The field of neural rendering and novel-view synthesis has been significantly advanced by the introduction of 3D Gaussian Splatting (3DGS).¹ This technique represents a paradigm shift from the prevalent implicit, volumetric ray-marching methods popularized by Neural Radiance Fields (NeRFs).³ Instead of using a neural network to map coordinates to color and density, 3DGS employs an explicit set of 3D Gaussian primitives. These primitives can be "splatted" or projected onto an image plane and rendered via a highly efficient rasterization pipeline. This explicit, rasterization-based approach has conferred several transformative advantages that have led to its rapid and widespread adoption: state-of-the-art visual quality, dramatically reduced training times, and, most notably, the ability to render complex scenes in real-time at high resolutions.²

However, the remarkable performance of canonical 3DGS implementations and the libraries they have inspired, such as the popular gsplat framework, is fundamentally tethered to custom-written CUDA kernels.⁷ These kernels are meticulously optimized to handle the most computationally intensive stage of the pipeline: the tile-based rasterization of millions of Gaussians. This reliance on bespoke CUDA code presents a significant barrier to entry for a broad segment of the research and development community. It demands specialized expertise in low-level GPU programming, complicates the development and debugging cycle, and reduces the portability of solutions across different hardware and software environments.¹⁰ For many practitioners, particularly in scientific domains where the primary focus is on modeling and analysis rather than graphics optimization, the need to compile and maintain custom C++/CUDA extensions is a prohibitive constraint.

This report provides a comprehensive, first-principles guide to designing and implementing a

high-performance Gaussian Splatting optimizer *entirely within the standard PyTorch framework*, eschewing the need for any custom CUDA code. The objective is to demystify the process and establish a consensus on best practices for CUDA-free implementations, drawing from pioneering open-source work such as the `hbb1/torch-splatting` repository.¹² The focus will be on mathematically sound, vectorizable algorithms and performance optimization strategies that leverage the full power of PyTorch's tensor computation engine. This guide is tailored to be generalizable to n-dimensions but will pay special attention to the practical application of representing 2D microscopy images as a non-negative sum of Gaussians, directly addressing the needs of scientific imaging analysis.

Section 1: Mathematical Foundations and Parametrization for Optimization

1.1 The n-Dimensional Gaussian Primitive

The fundamental building block of this representation is the n-dimensional Gaussian function. Each Gaussian acts as a primitive that describes a localized, continuous distribution of properties like color and opacity. In the familiar 3D case, these primitives manifest as arbitrarily oriented and scaled ellipsoids.³

The unnormalized probability density function of a multivariate Gaussian is given by the equation:

$$G(x) = \exp(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu))$$

Here, x is a point in n-dimensional space, μ is the n-dimensional mean vector representing the center of the Gaussian, and Σ is the $n \times n$ symmetric positive semi-definite covariance matrix. The covariance matrix Σ is of particular importance as it defines the shape, scale, and orientation of the Gaussian ellipsoid.¹³

1.2 Stable Parameter Representation in PyTorch

To optimize a set of Gaussians using gradient-based methods like those in PyTorch, each of

their properties must be represented by a differentiable tensor. A naive approach of directly optimizing the covariance matrix Σ is unstable because the optimization process does not guarantee that Σ will remain positive semi-definite, a necessary condition for a valid covariance matrix. Consequently, the consensus is to optimize a set of more fundamental geometric properties from which the covariance matrix can be constructed. This separation of concerns into distinct, interpretable parameters is not merely a mathematical convenience to enforce constraints; it is a crucial optimization strategy. It allows for the application of different learning rates and regularization schemes to geometric concepts like "size" and "orientation" independently, which has been found empirically to be vital for stable and efficient convergence.¹⁶

The standard set of optimizable parameters for a collection of N Gaussians is as follows:

- **Position (μ):** Represented directly as a torch.Tensor of shape $(N, 3)$ (for 3D). This tensor stores the XYZ coordinates of each Gaussian's center and is optimized in real space without any activation function.¹⁰
- **Scale (S):** To ensure the scale factors remain non-negative, they are optimized in log-space. A torch.Tensor of shape $(N, 3)$ stores the logarithm of the scaling factors along the three principal axes. The actual scale values are retrieved during the forward pass by applying an exponential function: `scales = torch.exp(log_scales)`. This prevents the scale from collapsing to zero or becoming negative during training.
- **Rotation (R):** Represented by unit quaternions, which provide a numerically stable and compact representation of 3D rotations, avoiding issues like gimbal lock. This is stored as a torch.Tensor of shape $(N, 4)$. During optimization, it is essential to periodically re-normalize the quaternions to ensure they remain on the unit hypersphere (`q = q / torch.linalg.norm(q, dim=-1, keepdim=True)`).
- **Opacity (α):** To constrain the opacity value to the valid range of $[0, 1]$, it is stored in logit-space as a torch.Tensor of shape $(N, 1)$. The final opacity is obtained by applying the sigmoid function: `opacity = torch.sigmoid(logits)`.
- **Color (c):** To model view-dependent effects, the color is not stored as a simple RGB value. Instead, it is represented by the coefficients of Spherical Harmonics (SH).³ This is stored in a torch.Tensor of shape $(N, C, (D+1)**2)$, where C is the number of color channels (e.g., 3 for RGB) and D is the degree of the SH basis. The 0th-degree coefficient represents the diffuse (view-independent) color component, while higher-order coefficients capture more complex anisotropic effects. While specialized libraries like `sphercart` exist for efficient SH evaluation, a pure PyTorch implementation can be written by directly implementing the SH basis functions.¹⁹

The following table summarizes this robust parametrization strategy.

Logical	PyTorch	Optimization	Activation	Purpose
---------	---------	--------------	------------	---------

Parameter	Representation	Space	Function	
Position	<code>torch.Tensor(N, 3)</code>	Real	None	Defines the center of the Gaussian ellipsoid.
Scale	<code>torch.Tensor(N, 3)</code>	Log	<code>torch.exp()</code>	Defines the extent of the ellipsoid along its principal axes. Ensures non-negativity.
Rotation	<code>torch.Tensor(N, 4)</code>	Unit Quaternions	Normalization	Defines the orientation of the ellipsoid. Numerically stable.
Opacity	<code>torch.Tensor(N, 1)</code>	Logit	<code>torch.sigmoid()</code>	Defines the transparency of the Gaussian. Constrains value to $[0, 1]$.
Color	<code>torch.Tensor(N, C, (D+1)**2)</code>	Real	None	Defines view-dependent color via Spherical Harmonic coefficients.

1.3 Code Implementation: Constructing the Covariance Matrix

With the scale and rotation parameters defined, the full 3D covariance matrix Σ for each

Gaussian can be constructed differentiably. The mathematical relationship is $\Sigma = RSS^T R^T$, where R is the 3×3 rotation matrix derived from the quaternion and S is a 3×3 diagonal scaling matrix.²¹ This construction can be implemented efficiently in PyTorch using vectorized operations.

Python

```
import torch
import torch.nn.functional as F

def build_covariance_from_params(scales, rotations):
    """
    Constructs the 3x3 covariance matrix from learned scale and rotation parameters.

    Args:
        scales (torch.Tensor): Tensor of scales of shape (N, 3).
        rotations (torch.Tensor): Tensor of quaternions of shape (N, 4).

    Returns:
        torch.Tensor: Tensor of covariance matrices of shape (N, 3, 3).
    """
    # Ensure quaternions are normalized
    norm_rotations = F.normalize(rotations, p=2, dim=1)

    # Convert quaternions to rotation matrices
    # PyTorch does not have a native batched quaternion to matrix conversion.
    # This can be implemented manually or using a library like pytorch3d.
    # For simplicity, here is a manual implementation:
    r, i, j, k = torch.unbind(norm_rotations, -1)
    two_s = 2.0 / (norm_rotations * norm_rotations).sum(-1)

    R = torch.zeros((norm_rotations.shape, 3, 3), device=scales.device)

    R[:, 0, 0] = 1.0 - two_s * (j * j + k * k)
    R[:, 0, 1] = two_s * (i * j - k * r)
    R[:, 0, 2] = two_s * (i * k + j * r)
    R[:, 1, 0] = two_s * (i * j + k * r)
    R[:, 1, 1] = 1.0 - two_s * (i * i + k * k)
    R[:, 1, 2] = two_s * (j * k - i * r)
    R[:, 2, 0] = two_s * (i * k - j * r)
    R[:, 2, 1] = two_s * (j * k + i * r)
```

```

R[:, 2, 2] = 1.0 - two_s * (i * i + j * j)

# Create scaling matrix S
S = torch.diag_embed(scales)

# Compute covariance: Sigma = R * S * S^T * R^T
# Since S is diagonal, S * S^T is a diagonal matrix with squared scales
L = R @ S
covariance = L @ L.transpose(1, 2)

return covariance

```

Section 2: The Differentiable Rendering Pipeline without Custom Kernels

The core of Gaussian Splatting is its differentiable rendering pipeline, which allows gradients from a 2D image loss to flow back to the 3D Gaussian parameters. This section details how to construct this pipeline in pure PyTorch.

2.1 Projecting 3D Gaussians to the 2D Image Plane

The first step in rendering is to project the 3D Gaussians from world space into the 2D camera's image plane. This is not a simple projection of the center point; the 3D covariance matrix Σ must also be projected to determine the shape of the 2D splat. The original paper uses a local affine approximation for the perspective projection, which results in the following formula for the 2D covariance matrix Σ' :

$$\Sigma' = JW\Sigma W^T J^T$$

Here, W is the 3×4 viewing transformation matrix (world-to-camera extrinsics), and J is the Jacobian of the projective transformation. This Jacobian captures how the perspective projection locally stretches and shears the space, and it depends on the camera intrinsics and the depth of the Gaussian's center in camera space.³ The official implementation confirms that this projection step can be performed in pure PyTorch, as evidenced by the

--convert_cov3D_python flag.⁷

2.2 The Rasterization Challenge: Why Official Implementations Rely on CUDA

While the projection mathematics are straightforward to implement with tensor libraries, the subsequent rasterization step is a significant performance bottleneck. This is the primary reason why high-performance implementations rely on custom CUDA kernels. A performant rasterizer must solve three key problems with massive parallelism:

1. **Culling/Binning:** For each small region of the image (a "tile"), it must efficiently identify the subset of potentially millions of Gaussians that overlap it.
2. **Sorting:** For each pixel, it must sort the overlapping Gaussians by depth to ensure correct front-to-back blending.
3. **Blending:** It must iterate through the sorted list for each pixel, accumulating color and opacity.

Custom CUDA kernels excel at this by using low-level hardware features. They can leverage fast on-chip shared memory to store tile data, use atomic operations for thread-safe memory writes, and employ highly optimized parallel sorting libraries like CUB, which contains primitives like device-wide radix sort.¹⁰ A pure Python implementation cannot access these hardware features directly and must rely on higher-level abstractions, which introduces overhead.

The algorithmic approach must therefore shift from a "pixel-centric" parallel model, where each thread processes a pixel, to a "tensor-centric" model. Instead of thinking "for each pixel, find its color," a PyTorch implementation must be structured as "for all pixels in a tile at once, compute their colors from all relevant Gaussians at once." This requires structuring the data into large tensors that can be processed with broadcasted operations, fully leveraging PyTorch's optimized backend for the heavy lifting.

2.3 A Pure PyTorch Tile-Based Rasterizer: Algorithm and Implementation

Despite the challenges, an efficient rasterizer can be built in pure PyTorch by adopting a tile-based approach and meticulously vectorizing every step. The following algorithm, inspired

by projects like hbb1/torch-splatting, outlines the process.¹²

1. **Frustum Culling:** The first step is to perform a coarse culling of Gaussians that are entirely outside the camera's viewing frustum. This can be done by checking if the Gaussian centers lie within the six planes of the view frustum. This significantly reduces the number of Gaussians that need to be processed in subsequent, more expensive steps.
2. **Project Means & Compute Bounding Boxes:** The 3D means (μ) of the surviving Gaussians are projected into 2D screen space using the full camera transformation matrix (world $\rightarrow$ camera $\rightarrow$ clip $\rightarrow$ screen). For each projected 2D Gaussian, a 2D bounding box is computed. The extent of this bounding box is determined by the projected 2D covariance matrix Σ' . A common heuristic is to use a 3-sigma confidence interval, which captures over 99% of the Gaussian's energy.
3. **Tile-Based Binning:** The screen is divided into a grid of tiles (e.g., 16x16 or 64x64 pixels).¹² A mapping is then created from each tile to a list of indices of the Gaussians whose 2D bounding boxes overlap it. This is a critical step to vectorize. One efficient method is to calculate the minimum and maximum tile coordinates covered by each Gaussian's bounding box. This information can then be used to construct an inverted index (a sparse tensor or dictionary-like structure) that maps each tile index to the relevant Gaussian indices.
4. **Per-Tile Rendering:** The final rendering is performed tile by tile. To avoid Python loops, tiles can be processed in batches. For each tile in the batch:
 - **Gather Gaussians:** Collect the parameters (2D mean, 2D covariance, color, opacity) of all Gaussians that overlap the current tile.
 - **Depth Sort:** Sort these gathered Gaussians based on their depth value (the z-component in camera space). This is crucial for correct alpha blending and can be done efficiently using torch.argsort.²³
 - **Evaluate on Pixel Grid:** Create a tensor representing the grid of pixel coordinates for the tile. For every pixel in this grid, evaluate the contribution of every sorted Gaussian. The core of this step is computing the Mahalanobis distance $(x-\mu')^T \Sigma'^{-1} (x-\mu')$ for all pixel-Gaussian pairs in a fully vectorized manner using tensor broadcasting.
 - **Alpha Blending:** With the per-pixel alpha values (α_i) for each sorted Gaussian, the final pixel color C is computed by blending front-to-back:
$$C = \sum_{i=1}^N c_i \alpha_i T_i \text{ where } T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

The transmittance term T_i can be computed very efficiently for all Gaussians simultaneously using torch.cumprod on the $(1 - \alpha)$ values.²⁴

A simplified code snippet for the vectorized alpha blending step demonstrates the principle:

Python


```
def alpha_composite(alphas, colors):
    """
    Performs vectorized alpha compositing.

    Args:
        alphas (torch.Tensor): Shape (N, H, W, 1), sorted front-to-back.
        colors (torch.Tensor): Shape (N, H, W, C), sorted front-to-back.

    Returns:
        torch.Tensor: The final composited image of shape (H, W, C).
    """
    # Calculate transmittance
    transmittance = torch.cumprod(1.0 - alphas + 1e-10, dim=0)
    # Prepend a 1.0 to the transmittance tensor for the first layer
    transmittance = torch.cat(
        [torch.ones_like(transmittance[:1]), transmittance[:-1]], dim=0
    )

    # Calculate weights for each layer
    weights = alphas * transmittance

    # Composite the colors
    final_color = torch.sum(weights * colors, dim=0)

    return final_color
```

2.4 Performance Optimization Strategies for the PyTorch Rasterizer

Making the pure PyTorch rasterizer performant requires careful attention to implementation details:

- **Aggressive Vectorization:** Avoid explicit Python for loops over pixels or Gaussians. Operations should be formulated using large tensor operations like `torch.matmul` and `torch.einsum` that can be dispatched to PyTorch's highly optimized backend.
- **Tile Size Tuning:** The choice of tile size involves a trade-off. Smaller tiles (e.g., 16x16) mean fewer Gaussians are processed per tile, but this may increase the overhead from Python logic if tiles are processed individually. Larger tiles (e.g., 64x64, as used in `torch-splatting`) reduce this overhead but increase the memory footprint and computation per tile.¹² The optimal size depends on the scene complexity and GPU

architecture.

- **Mixed-Precision Training:** Using `torch.cuda.amp` (Automatic Mixed Precision) can provide a significant speedup, especially for the matrix multiplication-heavy steps like covariance construction and projection, by leveraging the GPU's Tensor Cores.²⁶
- **Batched Tile Processing:** To manage memory usage for high-resolution images, instead of processing all tiles at once, they can be grouped into batches. This allows the model to fit within VRAM while still benefiting from vectorized computation within each batch.

Section 3: The Optimization Framework

3.1 Loss Functions for High-Fidelity Reconstruction

The optimization process aims to minimize the difference between the rendered image and the ground-truth training image. The standard loss function is a weighted combination of a pixel-wise L1 loss and a structural term, the D-SSIM (Structural Dissimilarity Index Measure).¹⁶

$$L = (1 - \lambda)L_1 + \lambda L_{D-SSIM}$$

- **L1 Loss (L1):** Calculated as `torch.nn.functional.l1_loss(rendered, ground_truth)`. It encourages the rendered image to match the ground truth in terms of absolute color values.
- **D-SSIM (LD-SSIM):** Calculated as $1.0 - \text{SSIM}(\text{rendered}, \text{ground_truth})$. The SSIM metric is designed to be more aligned with human perception of image quality than L1 or L2 losses. Including it in the loss function helps preserve high-frequency details and textures, preventing the overly smooth results that can arise from using L1 loss alone. A differentiable SSIM implementation is available in libraries like `torchmetrics`.

The weighting factor λ (typically set to 0.2) balances the influence of the two terms.¹⁶

3.2 Optimizer Configuration and Learning Rate Scheduling

The Adam optimizer is the de facto standard for training Gaussian Splatting models. A critical aspect of successful optimization is the use of different learning rates for the different groups of parameters, as they correspond to geometric properties with vastly different sensitivities

and scales.¹⁶ This is achieved in PyTorch by passing a list of parameter groups to the optimizer, each with its own

lr setting.

Furthermore, learning rate schedules are employed, particularly for the Gaussian positions. A common strategy is to use an initial warm-up period followed by an exponential decay. This allows for large-scale adjustments to the Gaussian positions early in training, followed by more precise fine-tuning in later stages.

The following table provides a robust set of starting hyperparameters, consolidated from community best practices.

Parameter Group	Optimizer	Initial Learning Rate	Scheduler Notes
Position	Adam	1.6×10^{-4}	Warm-up followed by exponential decay to 1.6×10^{-6} . ¹⁷
Scale	Adam	0.005	Fixed learning rate. ¹⁶
Rotation	Adam	0.001	Fixed learning rate. ¹⁶
Opacity	Adam	0.05	Fixed learning rate. ¹⁶
SH Features	Adam	0.0025	Fixed learning rate. ¹⁶

3.3 Adaptive Density Control: A Practical Trade-off for CUDA-Free Implementations

A key innovation of the original 3DGS paper is *adaptive density control*, a set of heuristics applied periodically during training to refine the set of Gaussians.³ This involves:

- **Cloning:** Duplicating small Gaussians in areas of high reconstruction error (indicated by

large positional gradients).

- **Splitting:** Splitting large Gaussians into two smaller ones, also in areas of high error.
- **Pruning:** Removing Gaussians that are nearly transparent (opacity below a threshold), as they contribute negligibly to the final image.

This process acts as an optimization accelerator. It dynamically allocates computational resources (Gaussians) to regions of the scene that are difficult to model, enabling the capture of fine details more efficiently. While the core optimization can converge with a fixed set of Gaussians, adaptive control often leads to faster convergence and a higher-quality final result with a more compact set of primitives.

However, implementing this in a CUDA-free context presents a significant performance challenge. The logic for identifying candidates, creating new tensors for the cloned/split Gaussians, and removing pruned Gaussians involves significant control flow and tensor resizing, which can be slow in Python. For this reason, some pure PyTorch implementations, like hbb1/torch-splatting, opt for a fixed number of Gaussians to maintain a simple and fast training loop.¹² This presents a strategic choice for the implementer: prioritize simplicity and predictable per-iteration speed by using a fixed set of Gaussians, or invest in a complex but carefully vectorized implementation of adaptive control for the potential of faster overall convergence. For initial implementations, starting with a fixed number of Gaussians is strongly recommended.

3.4 Regularization Techniques for Geometric Stability

Without careful control, the optimization process can produce undesirable artifacts, such as extremely elongated, "spiky" Gaussians that fit the training views well but generalize poorly.²⁷ Regularization can be introduced to promote more plausible geometries.

- **Scale Regularization:** To encourage Gaussians to be more isotropic (sphere-like), a loss term can be added to penalize large variances between the three scale components of each Gaussian. This can be a simple L2 loss on the log-scale parameters.
- **Depth Regularization:** If ground-truth or estimated depth maps are available, a loss can be applied to the rendered depth map to enforce geometric consistency. The depth of a pixel can be rendered differentiably by modifying the alpha blending step to accumulate depth values instead of colors.⁸

Section 4: Application and Extension to Scientific

Imaging

4.1 Specialization for 2D Microscopy Images

The general n -dimensional framework simplifies considerably when applied to the user's specific goal of representing 2D microscopy images.

- **Dimensionality Reduction:** The primitives are now 2D Gaussians. The position tensor becomes shape $(N, 2)$, the scale tensor is $(N, 2)$, and rotation simplifies to a single angle per Gaussian, which can be stored in a $(N, 1)$ tensor and used to construct a 2×2 rotation matrix.
- **Simplified Rendering:** The complex 3D-to-2D projection step is no longer needed. The "rendering" process becomes a direct evaluation of the 2D Gaussian mixture model on the 2D pixel grid of the image. The tile-based rasterization algorithm remains highly relevant for performance, as it efficiently limits the number of Gaussians that need to be evaluated for each pixel.

4.2 Enforcing the Non-Negative Sum Constraint

The requirement to represent an image as a "non-negative sum of Gaussians" translates to two simple constraints on the model's parametrization:

1. **Non-Negative Intensity:** Instead of an RGB color, each 2D Gaussian will have a single scalar intensity value. To ensure this value is non-negative, it should be optimized in log-space, analogous to the scale parameter. The actual intensity is then $\text{intensity} = \text{torch.exp}(\log_intensity)$.
2. **Zero Background:** The alpha compositing formulation naturally produces a sum of contributions. By ensuring the background color in the renderer is set to zero, the final image will be a sum of the non-negative Gaussian intensities.

This application reframes Gaussian Splatting from a computer graphics technique into a method for sparse signal representation. It can be understood as a form of differentiable dictionary learning. The image is reconstructed from a dictionary of "atoms," where each atom is a Gaussian basis function. Unlike fixed bases (e.g., Fourier or wavelet), the basis functions themselves—their position, shape, and orientation—are learned alongside their coefficients (intensities). This connection to a broader class of machine learning problems

opens the door to applying techniques from sparse coding, such as adding an L1 regularization penalty on the Gaussian intensities to encourage the model to find the most compact and meaningful representation of the underlying structures in the microscopy image.

4.3 Conceptual Extension to n-Dimensions

The pure PyTorch framework is conceptually straightforward to generalize to higher dimensions. The parameter tensors for position, scale, and rotation can be defined with n-dimensional vectors and matrices. The construction of the n-dimensional covariance matrix remains analogous. The primary challenge shifts from "rendering" in the graphical sense to "evaluation." In a general nD problem, the goal is typically to evaluate the Gaussian mixture model at a set of nD query points. This is a direct, vectorized computation in PyTorch, though the computational and memory costs will scale with the dimensionality.

Conclusion and Strategic Recommendations

The development of a Gaussian Splatting optimizer in pure PyTorch is not only feasible but also offers a portable, maintainable, and highly extensible alternative to CUDA-dependent implementations. The consensus for a successful implementation hinges on a set of core principles: stable parametrization using log-space for scales and opacities and quaternions for rotations; a fully vectorized, tile-based rasterizer that uses broadcasting and `torch.cumprod` to avoid Python loops; a combined L1 and D-SSIM loss function; and the use of the Adam optimizer with per-parameter learning rates and carefully designed schedules. For simplicity and performance, it is advisable to begin with a fixed number of Gaussians before considering the complexities of a vectorized adaptive density control scheme.

While a pure PyTorch implementation offers maximum accessibility, it is important to understand the performance landscape. For applications where the performance of the optimized PyTorch code is insufficient, an excellent middle ground is offered by libraries like `taichi-splatting`.²⁹ Taichi is a Python-embedded, just-in-time (JIT) compiler that can generate high-performance GPU kernels from Python syntax. This provides a path to near-CUDA performance without the need to write and compile C++ code, representing a powerful compromise between the simplicity of PyTorch and the raw speed of custom kernels.

Based on this analysis, the following strategic recommendations are proposed:

1. **Start with Pure PyTorch:** For maximum simplicity, portability, and ease of debugging,

the pure PyTorch approach detailed in this report is the recommended starting point. It is fully capable of producing high-quality results and is ideal for research and development, especially in scientific domains.

2. **Evaluate Taichi for Performance Scaling:** If, after optimization, the rendering or training speed of the pure PyTorch implementation becomes a bottleneck (e.g., for very high-resolution images or large-scale batch processing), taichi-splatting should be the next technology to evaluate. It offers a significant performance boost with a minimal increase in implementation complexity.
3. **Reserve Custom CUDA for Real-Time Graphics:** The development of custom CUDA kernels should be reserved for applications that require absolute state-of-the-art, real-time rendering performance on par with the original 3DGS paper. For many scientific analysis and modeling tasks, this level of optimization may be unnecessary.

The techniques and strategies discussed herein are part of a broader trend toward more accessible and powerful differentiable rendering frameworks.³⁰ By removing the dependency on low-level GPU programming, these pure-Python and JIT-based approaches are democratizing the ability to create and analyze complex n-dimensional data, paving the way for new applications in science, engineering, and beyond.

Works cited

1. TUM AI Lecture Series - The 3D Gaussian Splatting Adventure: Past, Present, Future (George Drettakis) - YouTube, accessed September 6, 2025, <https://www.youtube.com/watch?v=DjOqkVIIEGY>
2. 3D Gaussian Splatting for Real-Time Radiance Field Rendering - Inria, accessed September 6, 2025, <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
3. 3D Gaussian Splatting Introduction - Paper Explanation & Training on Custom Datasets with NeRF Studio Gsplits - LearnOpenCV, accessed September 6, 2025, <https://learnopencv.com/3d-gaussian-splatting/>
4. 3D Gaussian Splatting for Real-Time Radiance Field Rendering : r/singularity - Reddit, accessed September 6, 2025, https://www.reddit.com/r/singularity/comments/163jega/3d_gaussian_splatting_for_realtime_radiance_field/
5. [PDF] 3D Gaussian Splatting for Real-Time Radiance Field Rendering | Semantic Scholar, accessed September 6, 2025, <https://www.semanticscholar.org/paper/3D-Gaussian-Splatting-for-Real-Time-Radiance-Field-Kerbl-Kopanas/2cc1d857e86d5152ba7fe6a8355c2a0150cc280a>
6. New text To 3D Generation tool: 3D Gaussian Splatting for Real-Time Radiance Field Rendering : r/StableDiffusion - Reddit, accessed September 6, 2025, https://www.reddit.com/r/StableDiffusion/comments/16v4dvl/new_text_to_3d_generation_tool_3d_gaussian/
7. Original reference implementation of "3D Gaussian Splatting for Real-Time Radiance Field Rendering" - GitHub, accessed September 6, 2025, <https://github.com/graphdeco-inria/gaussian-splatting>

8. gsplat: An Open-Source Library for Gaussian Splatting - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2409.06765v1>
9. gsplat documentation, accessed September 6, 2025, <https://docs.gsplat.studio/>
10. Introduction to 3D Gaussian Splatting - Hugging Face, accessed September 6, 2025, <https://huggingface.co/blog/gaussian-splatting>
11. Is it possible to do gaussian splatting with a weak graphics card? : r/GaussianSplatting, accessed September 6, 2025, https://www.reddit.com/r/GaussianSplatting/comments/1h18zdi/is_it_possible_to_do_gaussian_splatting_with_a/
12. hbb1/torch-splatting: A pure pytorch implementation of 3D gaussian Splatting - GitHub, accessed September 6, 2025, <https://github.com/hbb1/torch-splatting>
13. Tungthanhlee/3DGS-pytorch - GitHub, accessed September 6, 2025, <https://github.com/Tungthanhlee/3DGS-pytorch>
14. Optimal Projection for 3D Gaussian Splatting - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2402.00752v2>
15. 2DGH: 2D Gaussian-Hermite Splatting for High-quality Rendering and Better Geometry Reconstruction - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2408.16982v1>
16. Official implementation of "Mesh-Based Adapting and Modification of Gaussian Splatting" - GitHub, accessed September 6, 2025, <https://github.com/waczjoan/gaussian-mesh-splatting>
17. 3D Gaussian Splatting for Real-Time Radiance Field Rendering - Gitee, accessed September 6, 2025, <https://gitee.com/trick12138/gaussian-splatting>
18. Rasterizing Gaussian Splats (The CUDA-less Way) - Of Shaders & Triangles - 42yeah, accessed September 6, 2025, <https://blog.42yeah.is/rendering/opengl/2023/12/20/rasterizing-splats.html>
19. PyTorch - sphericart documentation, accessed September 6, 2025, <https://sphericart.readthedocs.io/en/latest/pytorch-examples.html>
20. [2302.08381] Fast evaluation of spherical harmonics with sphericart - arXiv, accessed September 6, 2025, <https://arxiv.org/abs/2302.08381>
21. How to Render a Single Gaussian Splat? - Shi's blog, accessed September 6, 2025, https://shi-yan.github.io/how_to_render_a_single_gaussian_splat/
22. EP1918878B1 - Hierarchical tile-based rasterization algorithm - Google Patents, accessed September 6, 2025, <https://patents.google.com/patent/EP1918878B1/en>
23. limacv/GaussianSplattingViewer: Tiny Gaussian Splatting Viewer - GitHub, accessed September 6, 2025, <https://github.com/limacv/GaussianSplattingViewer>
24. 2D Gaussian Splatting implemented in python with a few ... - Colab, accessed September 6, 2025, <https://colab.research.google.com/drive/1qocID7HJ3-o0O1R8cvV3PxLhoDCMsH8W?usp=sharing>
25. 3D Gaussian Splatting In Practice, accessed September 6, 2025, https://3dgstutorial.github.io/3dv_part2.pdf
26. Can you do mixed precision with just the forward pass and not the loss? - PyTorch Forums, accessed September 6, 2025, <https://discuss.pytorch.org/t/can-you-do-mixed-precision-with-just-the-forward>

[-pass-and-not-the-loss/175868](#)

27. Splatfacto - nerfstudio, accessed September 6, 2025, <https://docs.nerf.studio/nerfology/methods/splat.html>
28. gaussian-splatting - PyPI, accessed September 6, 2025, <https://pypi.org/project/gaussian-splatting/>
29. taichi-splatting - PyPI, accessed September 6, 2025, <https://pypi.org/project/taichi-splatting/>
30. renderer - PyTorch3D, accessed September 6, 2025, <https://pytorch3d.org/docs/renderer>
31. Kaolin Tools for 3D Deep Learning Research - NVIDIA Developer, accessed September 6, 2025, <https://developer.nvidia.com/kaolin>
32. PyTorch3D · A library for deep learning with 3D data, accessed September 6, 2025, <https://pytorch3d.org/>